



25 PYSPARK DATA QUALITY CHECKS

THAT PREVENT PRODUCTION DISASTERS



 Trusted Data
  Reliable Pipelines
  Better Decisions
  Happier Stakeholders

1
 NULL / MISSING VALUE CHECK
 
 Detect nulls in important columns.


```
df.filter(col("col").isNull()).count()
```

2
 PRIMARY KEY UNIQUENESS CHECK
 
 Ensure primary key is unique.


```
df.groupBy("id").count().filter("count > 1")
```

3
 DUPLICATE RECORD DETECTION
 
 Identify duplicate records.


```
df.groupBy(df.columns).count().filter("count > 1")
```

4
 REFERENTIAL INTEGRITY CHECK
 
 Validate foreign key exists in parent.


```
df.join(parent_df, "fk_id", "left").filter(parent_df.id.isNull())
```

5
 DATA TYPE VALIDATION
 
 Ensure column has expected data type.


```
df.filter(~col("age").cast("int").isNull())
```

6
 NUMERIC RANGE CHECK
 
 Validate numeric values are in acceptable range.


```
df.filter(col("age") < 0 | col("age") > 120)
```

7
 STRING LENGTH VALIDATION
 
 Check string length is within limits.


```
df.filter((length(col("name")) < 2) | (length(col("name")) > 100))
```

8
 REGEX PATTERN VALIDATION
 
 Validate values using regex patterns.


```
df.filter(~col("email").rlike("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"))
```

9
 ALLOWED VALUES / DOMAIN CHECK
 
 Ensure values exist in allowed list.


```
allowed = ["A", "B", "C"]
df.filter(~col("status").isin(allowed))
```

10
 BUSINESS RULE CONSISTENCY
 
 Validate business rules (aggregates, ratios, etc.).


```
df.filter(col("total") != col("qty") * col("price"))
```

11
 CROSS-COLUMN CONSISTENCY
 
 Check logical relation between columns.


```
df.filter(col("start_date") > col("end_date"))
```

12
 TIMELINESS / FRESHNESS CHECK
 
 Check data is within expected time window.


```
from pyspark.sql.functions import current_date, datediff
df.filter(datediff(current_date(), col("event_date")) > 1)
```

13
 COMPLETENESS CHECK (PARTITIONS)
 
 Ensure all expected partitions/files exist.


```
from pyspark.sql.functions import input_file_name
df.select(input_file_name()).distinct()
```

14
 VOLUME CHECKS AGAINST THRESHOLDS
 
 Validate row counts against historical range.


```
cnt = df.count()
# compare with min/max threshold
```

15
 DISTRIBUTION CHECKS
 
 Check data distribution (mean, median, std dev).


```
df.select("value").summary("count", "mean", "stddev", "min", "max").show()
```

16
 OUTLIER DETECTION FOR NUMERIC VALUES
 
 Detect outliers using IQR or Z-score.


```
from pyspark.ml.feature import VectorAssembler
# or custom IQR logic
```

17
 SCHEMA DRIFT DETECTION
 
 Detect new, removed or changed columns.


```
# compare current schema with expected schema
```

18
 DUPLICATE FILE INGESTION CHECK
 
 Detect same file ingested multiple times.


```
df.groupBy("file_name").count().filter("count > 1")
```

19
 NEGATIVE VALUE CHECKS
 
 Validate negative values where not allowed.


```
df.filter(col("amount") < 0)
```

20
 PERCENTAGE / TOTAL CONSISTENCY
 
 Check percentages add up to 100% (or valid range).


```
df.groupBy("group").agg(F.sum("pct").alias("total_pct")).filter(col("total_pct") != 100)
```

21
 HIERARCHY VALIDATION
 
 Validate hierarchy relationships.


```
df.join(parent, df.parent_id == parent.id, "left").filter(parent.id.isNull())
```

22
 AUDIT COLUMN CONSISTENCY
 
 Validate audit columns (created_at, updated_at, etc.).


```
df.filter(col("updated_at") < col("created_at"))
```

23
 CROSS-SYSTEM CONSISTENCY CHECK
 
 Validate counts between multiple systems.


```
cnt1 = df1.count()
cnt2 = df2.count()
# compare counts
```

24
 REFERENCE DATA VALIDATION
 
 Ensure reference/master data is up-to-date.


```
df.join(ref_df, "ref_id", "left").filter(ref_df.ref_id.isNull())
```

25
 CHECKSUM / HASH CONSISTENCY
 
 Validate checksum/hash to ensure data integrity.


```
from pyspark.sql.functions import md5, concat_ws
df.withColumn("hash", md5(concat_ws(" ", *df.columns)))
```



BEST PRACTICES

- Automate checks in pipelines
- Fail fast & alert early
- Monitor trends over time
- Keep rules in version control
- Document expectations clearly

USEFUL PYSPARK FUNCTIONS

```
isNull(), isNotNull()
count(), distinct(), groupBy()
filter(), where()
length(), rlike(), cast()
agg(), sum(), avg(), stddev()
datediff(), current_date()
```

KEY TAKEAWAY

Great data quality doesn't happen by accident — it happens by validation!

Implement these 25 checks, automate them, and build data pipelines that your organization can trust.



<https://medium.com/@one.step.analytics.on.data/25-pyspark-data-quality-checks-that-prevent-production-disasters-47ccedb6620c>

3/17

✓ Bad data is expensive. Good data is an advantage.
Validate early. Validate always. Build trust in every pipeline.

25 PySpark Data Quality Checks That Prevent Production Disasters



Anurag Sharma · 5 min read · Jun 22, 2026



10



4



Data is the lifeblood of modern organizations, but poor data quality can turn that lifeblood into poison. According to industry reports, bad data costs companies millions every year in lost opportunities, faulty decisions, and compliance headaches. As data engineers, one of our most critical responsibilities is to ensure data is accurate, complete, consistent, and reliable.

In this article, we'll explore **25 essential data quality checks** that every data engineer should know — inspired by proven industry practices. For each check, I'll provide a practical PySpark implementation using Spark DataFrames, which is the go-to tool for scalable data processing.

A home for all
your stories.

Write about
what fascinates
you on Medium.

Learn more

Whether you're building ETL pipelines, maintaining data lakes, or powering analytics platforms, these checks will help you catch issues early and build

trust in your data.



1. Null Value / Missing Check

Ensure critical columns have no null or missing values.

```
from pyspark.sql.functions import col, count, when

def check_nulls(df, columns):
    null_counts = df.select([count(when(col(c).isNull(), c)).alias(c) for c in columns])
    return null_counts

# Usage
null_df = check_nulls(df, ["id", "name", "email"])
null_df.show()
```

2. Primary Key Uniqueness Check

Primary keys must be unique with no duplicates.

```
def check_primary_key_uniqueness(df, pk_columns):
    duplicates = df.groupBy(pk_columns).count().filter(col("count") > 1)
    return duplicates

# Example
```



```
dupe_pks = check_primary_key_uniqueness(df, ["customer_id"])
dupe_pks.show()
```

3. Duplicate Record Detection

Identify completely duplicate rows.

```
def check_duplicate_records(df):
    total_rows = df.count()
    unique_rows = df.dropDuplicates().count()
    duplicates = total_rows - unique_rows
    return duplicates, df.dropDuplicates()
```

4. Referential Integrity Check

Validate foreign keys exist in the parent table.

```
def check_referential_integrity(fact_df, dim_df, fk_col, pk_col):
    invalid = fact_df.join(dim_df, fact_df[fk_col] == dim_df[pk_col], "left_anti")
    return invalid
```

5. Data Type Validation

Ensure columns match expected data types.

```
def check_data_types(df, schema_dict):
    issues = []
    for col_name, expected_type in schema_dict.items():
        actual_type = dict(df.dtypes)[col_name]
```

```

if actual_type != expected_type:
    issues.append((col_name, actual_type, expected_type))
return issues

```

6 NUMERIC RANGE CHECK  Validate numeric values are in acceptable range. <code>df.filter(col("age") < 0) col("age") > 120)</code>	7 STRING LENGTH VALIDATION  Check string length is within limits. <code>df.filter((length(col("name")) < 2) (length(col("name")) > 100))</code>	8 REGEX PATTERN VALIDATION  Validate values using regex patterns. <code>df.filter(~col("email").rlike("^[A-Za-z0-9._%+-]+@A-Za-z0-9.-]+\.[A-Za-z]{2,}\$"))</code>	9 ALLOWED VALUES / DOMAIN CHECK  Ensure values exist in allowed list. <code>allowed = ["A", "B", "C"] df.filter(~col("status").isin(allowed))</code>	10 BUSINESS RULE CONSISTENCY  Validate business rules (aggregates, ratios, etc.). <code>df.filter(col("total") != col("qty") * col("price"))</code>
--	--	--	--	--

6. Numeric Range Check

Validate values fall within acceptable ranges.

```

def check_numeric_range(df, column, min_val, max_val):
    invalid = df.filter((col(column) < min_val) | (col(column) > max_val))
    return invalid

```

7. String Length Validation

Check length constraints on string fields.

```

def check_string_length(df, column, min_len=1, max_len=255):
    invalid = df.filter(
        (length(col(column)) < min_len) | (length(col(column)) > max_len)
    )
    return invalid

```

8. Regex Pattern Validation

Validate formats like email, phone, or custom patterns.

```
def check_regex_pattern(df, column, regex):  
    return df.filter(~col(column).rlike(regex))
```

Email example:

```
email_regex = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'  
invalid_emails = check_regex_pattern(df, "email", email_regex)
```

9. Allowed Values / Domain Validation

Ensure values belong to a predefined set.

```
def check_allowed_values(df, column, allowed_set):  
    return df.filter(~col(column).isin(allowed_set))
```

10. Business Rule / Consistency Check

Validate aggregate rules (e.g., order total consistency).

```
def check_business_rule(df):
```

```
return df.filter(col("total_amount") != col("unit_price") * col("quantity"))
```

11 CROSS-COLUMN CONSISTENCY  Check logical relation between columns. <code>df.filter(col("start_date") > col("end_date"))</code>	12 TIMELINESS / FRESHNESS CHECK  Check data is within expected time window. <code>from pyspark.sql.functions import current_date, datediff</code> <code>df.filter(datediff(current_date(), col("event_date")) > 1)</code>	13 COMPLETENESS CHECK (PARTITIONS)  Ensure all expected partitions/files exist. <code>from pyspark.sql.functions import input_file_name</code> <code>df.select(input_file_name()).distinct()</code>	14 VOLUME CHECKS AGAINST THRESHOLDS  Validate row counts against historical range. <code>cnt = df.count()</code> <code># compare with min/max threshold</code>	15 DISTRIBUTION CHECKS  Check data distribution (mean, median, std dev). <code>df.select("value").summary("count", "mean", "stddev", "min", "max").show()</code>
--	---	--	--	--

11. Cross-Column Consistency Checks

Check logical relationships between columns.

```
def check_cross_column_consistency(df):
    # Example: end_date > start_date
    return df.filter(col("end_date") < col("start_date"))
```

12. Timeliness / Freshness Check

Ensure data is not stale.

```
from pyspark.sql.functions import current_timestamp, datediff

def check_data_freshness(df, timestamp_col, max_hours=24):
    stale = df.filter(
        datediff(current_timestamp(), col(timestamp_col)) * 24 > max_hours
    )
    return stale
```

13. Completeness Check (Partitions)

Validate expected partitions exist.

```
def check_partitions(df, partition_cols, expected_partitions):  
    existing = df.select(partition_cols).distinct()  
    missing = [p for p in expected_partitions if p not in existing]  
    return missing
```

14. Volume / Row Count Checks

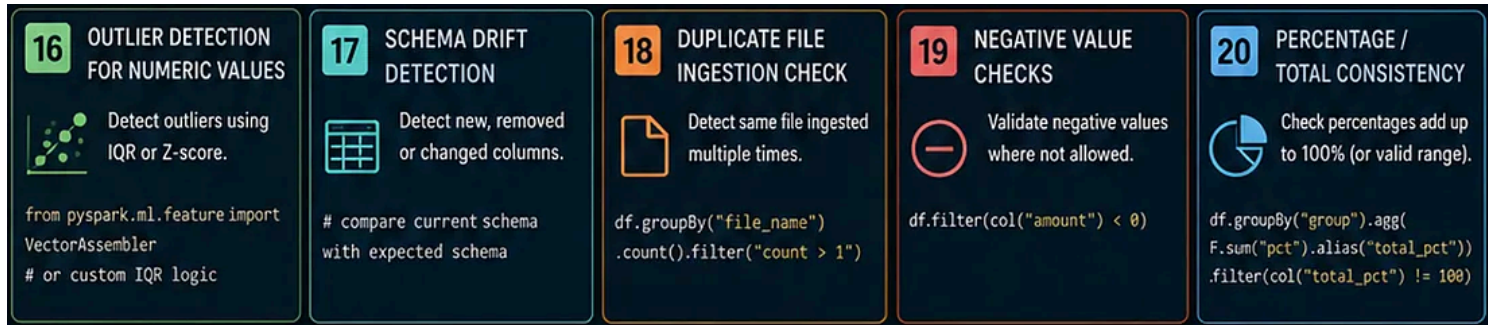
Monitor unexpected drops or spikes in record volume.

```
def check_volume(df, expected_min, expected_max):  
    count = df.count()  
    return count < expected_min or count > expected_max
```

15. Distribution Checks

Validate statistical distribution of numeric fields.

```
def check_distribution(df, column, expected_mean, expected_std, tolerance=0.1):  
    stats = df.select(mean(column).alias("mean"), stddev(column).alias("std")).c  
    return abs(stats["mean"] - expected_mean) > tolerance or abs(stats["std"] -
```

16. Outlier Detection for Abnormal Values

Detect statistical outliers.

```
from pyspark.ml.feature import StandardScaler, VectorAssembler

def detect_outliers(df, column, threshold=3.0):
    assembler = VectorAssembler(inputCols=[column], outputCol="features")
    scaler = StandardScaler(inputCol="features", outputCol="scaledFeatures")
```

17. Schema Drift Detection

Detect changes in schema over time.

```
def detect_schema_drift(current_schema, expected_schema):
    return set(current_schema) != set(expected_schema)
```

18. Duplicate File Ingestion Check

Prevent processing the same file multiple times.

```
def check_duplicate_files(processed_files, new_files):
    return set(new_files) & set(processed_files)
```

19. Negative Value Checks

Ensure business-critical fields are non-negative.

```
def check_negative_values(df, columns):
    return df.filter(reduce(lambda x, y: x | y, [col(c) < 0 for c in columns]))
```

20. Percentage / Total Consistency

Validate percentages add up to 100%.

```
def check_percentage_total(df, percent_columns):
    return df.filter(abs(df.select(sum(percent_columns)).collect()[0][0] - 100)
```

21 HIERARCHY VALIDATION  Validate hierarchy relationships. <pre>df.join(parent, df.parent_id == parent.id, "left").filter(parent.id.isNull())</pre>	22 AUDIT COLUMN CONSISTENCY  Validate audit columns (created_at, updated_at, etc.). <pre>df.filter(col("updated_at") < col("created_at"))</pre>	23 CROSS-SYSTEM CONSISTENCY CHECK  Validate counts between multiple systems. <pre>cnt1 = df1.count() cnt2 = df2.count() # compare counts</pre>	24 REFERENCE DATA VALIDATION  Ensure reference/master data is up-to-date. <pre>df.join(ref_df, "ref_id", "left").filter(ref_df.ref_id.isNull())</pre>	25 CHECKSUM / HASH CONSISTENCY  Validate checksum/hash to ensure data integrity. <pre>from pyspark.sql.functions import md5, concat_ws df.withColumn("hash", md5(concat_ws(" ", *df.columns)))</pre>
---	---	---	---	---

21. Hierarchy Validation

Validate hierarchical relationships (e.g., region > country).

```
def check_hierarchy(df, parent_col, child_col):  
    return df.filter(~col(child_col).startswith(col(parent_col))) # simplified
```

22. Audit Column Consistency

Validate audit fields like created_at and updated_at.

```
def check_audit_columns(df):  
    return df.filter(col("updated_at") < col("created_at"))
```

23. Cross-System Consistency Check

Compare data across different sources.

```
def cross_system_validation(source1_df, source2_df, join_key):  
    discrepancies = source1_df.join(source2_df, join_key, "outer").filter(  
        # add conditions for mismatch  
    )  
    return discrepancies
```

24. Reference Data Validation

Ensure lookup/reference data is consistent.

```
def validate_reference_data(df, ref_df, join_col):  
    invalid = df.join(ref_df, join_col, "left_anti")  
    return invalid
```

25. Checksum / Hash Consistency

Validate data integrity using hashes.

```
from pyspark.sql.functions import md5, concat_ws  
  
def check_hash_consistency(df, columns):  
    df = df.withColumn("hash", md5(concat_ws("|", *columns)))  
    # Compare with stored hash  
    return df
```

Why Data Quality Checks Matter

Implementing these checks proactively:

- Prevents bad data from reaching downstream consumers
- Reduces production incidents
- Improves analytics accuracy
- Ensures regulatory compliance (GDPR, SOX, etc.)
- Builds trust across the organization

Best Practices for DQ in PySpark

1. Automate everything — Integrate checks into your Airflow/Dagster pipelines.

2. **Fail fast** — Use Great Expectations or a custom DQ framework for assertions.
3. **Log and alert** — Push violations to monitoring systems (Prometheus, Datadog).
4. **Version your rules** — Store DQ rules in a central repository.
5. **Profile continuously** — Use Spark's built-in profiling + custom metrics.

Data Engineering

Data Engineer

Pyspark

Big Data

Data Quality

**Written by Anurag Sharma**[Edit profile](#)

125 followers · 5 following

Data Engineering Specialist with 10+ exp. Passionate about optimizing pipelines, data lineage, and Spark performance and sharing insights to empower data pros!